

Git 简明指南

从零开始掌握版本控制

计算机专业学习笔记

2026 年 3 月 22 日

目录

1 引言：为什么需要 Git ?	2
2 预备知识：Git 的四个区域	2
3 手把手实战：一次完整的推送流程	3
3.1 场景设定	3
3.2 错误示范（我们在终端里看到的情况）	3
3.3 正确步骤	3
4 常见错误与纠正	4
4.1 命令拼写错误	4
4.2 忘掉 git add 直接 git commit	5
4.3 推送前忘记拉取远程更新	5
4.4 提交信息写错了怎么办?	5
5 进阶技巧：让 Git 更聪明	5
5.1 使用 .gitignore 忽略垃圾文件	5
5.2 SSH 密钥免密登录	5
5.3 设置上游跟踪（-u 的妙用）	6
5.4 查看历史	6
6 总结：记住这三步	6
7 参考资源	6

1 引言：为什么需要 Git ？

想象一下，你正在写一篇论文：

- 第一次修改后保存为 论文 _v1.doc；
- 第二次修改后保存为 论文 _v2.doc；
-
- 第 N 次后桌面上全是 论文 _ 最终版.doc、论文 _ 最终版 2.doc、论文 _ 打死也不再改.doc。

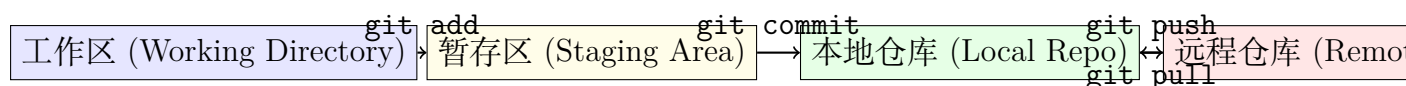
这不仅是管理噩梦，更是合作的灾难。Git 就像一个“时间旅行机”+“团队同步器”：

- 它可以记录每一次修改，随时回到过去；
- 它可以让你和他人并行工作，自动合并代码；
- 它让你的代码在云端安全存储。

但 Git 的命令行初学者常常感到困惑。本文将通过一个真实的案例（来自我们遇到的小问题）来手把手教你 Git 的基本操作，并带你走出误区。

2 预备知识：Git 的四个区域

Git 的核心概念可以类比为厨房里的四个“区域”：



- **工作区**：你正在编辑文件的真实文件夹（比如你在 VS Code 里写代码的地方）。
- **暂存区**：一个临时存放区，就像购物车，你想把哪些修改放进下一次“快照”里，就用 `git add` 放进去。
- **本地仓库**：存放在你电脑 `.git` 文件夹里的所有历史记录。每次 `git commit` 都会生成一个唯一的快照（类似游戏存档）。
- **远程仓库**：放在 GitHub、GitLab 等服务器上的备份，方便协作和异地存取。

**** 关键 ****：三个命令对应三步动作：

`git add` → `git commit` → `git push`

3 手把手实战：一次完整的推送流程

3.1 场景设定

你已经在 GitHub 上创建了一个仓库，并在本地克隆了下来。你新增了几个文件（比如 Template.tex、Template.pdf 等），现在想把它们推送到远程。

3.2 错误示范（我们在终端里看到的情况）

```
1 $ git push .
2 Everything up-to-date
3 $ git add .
4 $ git merge -u origin main
5 error: unknown switch `u'
6 $ git commit -u origin main
7 error: pathspec 'origin' did not match any file(s)
8 $ git commit -u
9 [main 48265d3] Add more notes.
10 2 files changed, 0 insertions(+), 0 deletions(-)
11 create mode 100644 How-to-get-and-build-an-opensource-project.pdf
12 create mode 100644 install-nvidia-smi.pdf
13 $ git merge
14 Already up to date.
```

问题出在哪？

1. `git push .` 中的 `.` 不是一个有效的远程仓库名（默认远程叫 `origin`）。
2. 新文件没有被 `git add`，所以 `git commit` 提交的是旧内容（实际上是之前已经存在并提交过的两个 PDF，所以第二次提交就提示“没变化”）。
3. `git merge -u` 和 `git commit -u` 都是错误的命令用法，选项不存在。
4. 最后虽然生成了一个提交 48265d3，但这是旧文件的重复提交，新文件依然未被跟踪。

3.3 正确步骤

1. 查看当前状态

```
1 $ git status
2 On branch main
3 Your branch is up to date with 'origin/main'.
```

```
4
5 Untracked files:
6   (use "git add <file>..." to include in what will be committed)
7       Template.aux
8       Template.log
9       Template.out
10      Template.pdf
11      Template.tex
12      Template.toc
13      git-push-tutorial.pdf
```

“Untracked files”意味着这些文件从未被 Git 管理过，它们还没进入“购物车”。

2. 添加需要跟踪的文件（把菜放进购物车）

```
1 $ git add Template.tex Template.pdf git-push-tutorial.pdf
```

或者你想一次添加所有新文件：`git add .`。但通常我们不希望把 LaTeX 编译生成的临时文件（`.aux`, `.log`, `.out`, `.toc`）纳入版本控制，所以最好手动添加或使用 `.gitignore`。

3. 提交（把购物车里的菜做成一道菜，存进冰箱）

```
1 $ git commit -m "Add LaTeX template and git push tutorial PDF"
```

这里 `-m` 后面是提交信息，用于说明这次改动的目的。

4. 推送到远程（把菜谱分享到云上）

```
1 $ git push origin main
```

如果这是第一次推送该分支，可以用 `git push -u origin main`，这样以后直接 `git push` 即可。

执行后，在 GitHub 上就能看到新文件了。

4 常见错误与纠正

4.1 命令拼写错误

- `git merge -u origin main` → 应该用 `git pull origin main`（拉取并合并）或 `git fetch`; `git merge origin/main`。
- `git commit -u` → `git commit -m "message"`。 `-u` 是 `git push` 的选项，不能混用。

4.2 忘掉 git add 直接 git commit

```
1 $ git commit -m "update"
2 On branch main
3 nothing to commit, working tree clean
```

这是因为没有先用 git add 把改动放进暂存区。记住：commit 只是把暂存区的快照存下来。

4.3 推送前忘记拉取远程更新

如果你和他人协作，远程可能已经有了新提交。这时直接 git push 会被拒绝。正确做法：

```
1 $ git pull origin main    # 拉取并合并
2 # 如果有冲突，解决冲突后 add 和 commit
3 $ git push origin main
```

4.4 提交信息写错了怎么办？

如果还没推送，可以用 git commit --amend -m "新的信息" 修改最后一次提交的信息。

5 进阶技巧：让 Git 更聪明

5.1 使用 .gitignore 忽略垃圾文件

LaTeX 编译时会产生一堆临时文件，每次 git status 都看到它们很烦。我们可以创建一个 .gitignore 文件，告诉 Git 忽略这些文件。

```
1 $ echo "*.aux"
2 *.log
3 *.out
4 *.toc" > .gitignore
5 $ git add .gitignore
6 $ git commit -m "Ignore LaTeX temporary files"
```

以后再编译 LaTeX，Git 会自动忽略这些文件。

5.2 SSH 密钥免密登录

如果你每次 git push 都要输入密码，可以配置 SSH 密钥并加入 ssh-agent：

```
1 $ eval "$(ssh-agent -s)"
2 $ ssh-add ~/.ssh/id_ed25519 # 输入一次密码，当前终端会话有效
```

或者将密钥加入系统的 Keychain (Mac) 或 Windows Credential Manager。

5.3 设置上游跟踪 (-u 的妙用)

第一次推送时使用 `git push -u origin main`，以后只需输入 `git push` 或 `git pull`，Git 就知道要推送到哪个远程分支。

5.4 查看历史

```
1 $ git log --oneline --graph # 以图形化方式查看提交历史
```

6 总结：记住这三步

不管多复杂的 Git 操作，最核心的就是这三板斧：

1. 修改文件
2. `git add` (放进暂存区)
3. `git commit` (形成快照)
4. `git push` (分享到远程)

每次操作前，先用 `git status` 看看当前状态，它就像一个 GPS，告诉你现在在哪、接下来该干什么。

7 参考资源

- 《Pro Git》免费电子书：<https://git-scm.com/book/zh/v2>
- GitHub 官方文档：<https://docs.github.com/zh>
- 可视化 Git 练习网站：<https://learngitbranching.js.org/>

祝你在 *Git* 的时光旅行中畅通无阻！